

Software Testing

Chapter Study Notes

Software Engineering
Ian Sommerville — 10th Edition | Chapter 8

Table of Contents

- Topic 1 — What is Program Testing?
- Topic 2 — Manual vs Automated Testing & V&V;
- Topic 3 — Functional & Non-Functional Testing
- Topic 4 — Validation and Defect Testing
- Topic 5 — Testing Process Goals
- Topic 6 — V&V; Confidence
- Topic 7 — Inspections and Testing — Static vs Dynamic
- Topic 8 — Software Inspections
- Topic 9 — Advantages of Inspections
- Topic 10 — Stages of Testing Overview
- Topic 11 — Development Testing
- Topic 12 — Stages of Development Testing: Unit, Component & System
- Topic 13 — Unit Testing & Object Class Testing
- Topic 14 — Automated Testing
- Topic 15 — Choosing Unit Test Cases
- Topic 16 — Equivalence Partitioning
- Topic 17 — Testing Guidelines

Topic 1: What is Program Testing?

"Testing is the process of executing a program with the intent of finding errors — using sample data to show what a program does and detect anomalies."

Two Core Purposes

- Showing what a program is doing — confirming it works correctly
- Detecting anomalies — finding things that go wrong

Testing Uses Sample Data

You cannot test every possible input — instead you use representative sample data to cover as many scenarios as possible.

How Much Testing is Required?

Software Type	Testing Requirement
Custom Software	At least 1 test for every requirement in the requirements document
Generic Software	Tests for every feature present in the product release

What Does Testing Look For?

Testing roots out undesirable system behavior such as:

- System crashes
- Unwanted interactions with other systems
- Incorrect computations
- Data corruption

Exam Takeaway: Testing uses sample data to show correct behavior and detect anomalies. Custom software needs at least one test per requirement; generic software needs tests for every feature.

Topic 2: Manual vs Automated Testing & V&V:

Manual vs Automated Testing

1. Manual Testing

- A human physically runs the program and checks results
- Time consuming and prone to human error
- Still used in exploratory and usability testing

2. Automated Testing

- Test cases are written as code and executed automatically
- Example framework: JUnit in Java
- Much faster and more reliable than manual testing
- Sommerville recommends automated testing wherever possible

Testing is Part of V&V;

Testing is not standalone — it is part of the broader process called Verification and Validation (V&V;).

Question	Type	Meaning
Are we building the RIGHT product?	Validation	Does software do what the user actually needs?
Are we building the product RIGHT?	Verification	Does software conform to its specification?

Black-Box vs White-Box Testing

1. Black-Box Testing

- Testing the system without looking at the internal code
- Tester only knows inputs and expected outputs
- Tests the system as a whole from the outside

Example: Testing a login page by just entering usernames and passwords

2. White-Box Testing

- Testing each component separately by looking at the internal code
- Tester knows the internal structure and logic
- Tests specific paths, branches, and conditions inside the code

Example: Testing each individual function/method in the code

Exam Takeaway: Testing can be manual or automated (JUnit). It is part of V&V; — Validation asks 'right product?' and Verification asks 'built right?'. Testing is classified as Black-Box (external) or White-Box (internal, component by component).

Topic 3: Functional & Non-Functional Testing

Two Types Based on Requirements

1. Functional Testing

- Testing whether the system does what it is supposed to do
- Checks each functionality described in the requirements
- Verifies that every feature works correctly as specified

Example: Testing that a Login button actually logs the user in, Search bar returns correct results, Checkout completes an order

2. Non-Functional Testing

- Testing the quality attributes of the system — not just what it does, but how well it does it
- Checks things that are not specific features but overall system behavior

Type	What it Checks
Performance testing	How fast does the system respond under normal load?
Scalability testing	Can the system handle increasing numbers of users/data?
System recovery testing	Does the system recover correctly after a crash or failure?
Security testing	Is the system protected against unauthorized access?
Usability testing	Is the system easy to use?

Exam Takeaway: Functional testing checks WHAT the system does — each feature against requirements. Non-functional testing checks HOW WELL the system performs — covering performance, scalability, recovery and more.

Tonic 4: Validation and Defect Testing

The Two Goals of Testing

1. Validation Testing

"A successful test shows that the system operates as intended."

- Goal: confirm to the developer and customer that the system meets user requirements
- Test cases reflect normal, expected use of the system
- Success = system behaves as expected

2. Defect Testing

"A successful test is a test that makes the system perform incorrectly and so exposes a defect."

- Goal: find bugs and anomalies in the system
- You are trying to deliberately break the system
- Test cases can be obscure and unusual — they don't need to reflect normal use
- Success = you found a defect

Example: Entering extreme values, unexpected characters, or impossible combinations to expose a crash

Key Difference

	Validation Testing	Defect Testing
Goal	Confirm system works	Find bugs/defects
Success means	System behaved correctly	System failed — bug exposed
Test cases	Reflect normal use	Can be obscure and unusual

Important note from Sommerville: No hard boundary is defined between validation testing and defect testing. Both can overlap — the same test case can simultaneously validate correct behavior AND expose a defect.

Exam Takeaway: Testing has two goals — Validation (prove it works) and Defect testing (prove it breaks). In validation, success = system works; in defect testing, success = system fails and exposes a bug.

Topic 5: Testing Process Goals

"To demonstrate to the developer and the system customer that the software meets its requirements."

Goal 1: Validation Testing

- A successful test = the system operates as intended
- Demonstrates correctness to two audiences — the developer AND the customer
- Focus is on confidence — giving stakeholders confidence the system works

Goal 2: Defect Testing

"To discover faults or defects in the software where its behavior is incorrect or not in conformance with its specification."

- A successful test = the test makes the system perform incorrectly — exposing a defect
- Test cases can be deliberately obscure
- They do NOT need to reflect how the system is normally used
- Focus is on finding failures — not confirming success

The Critical Mindset Difference

	Validation Testing	Defect Testing
Mindset	I want to show it works	I want to show it breaks
Test designer thinks	What does the system do correctly?	How can I make it fail?
Successful test means	System passed	System failed — bug found

Exam Takeaway: The testing process has two formal goals — Validation (demonstrate system meets requirements) and Defect testing (discover incorrect behavior). These two goals drive all testing activities.

Topic 6: V&V: Confidence

"The aim of V&V; is to establish confidence that the system is 'fit for purpose'."

Fit for purpose means the system does what it needs to do well enough for its intended use. It does not mean zero defects — it means the system is good enough for its context.

3 Factors That Determine How Much Confidence is Needed

1. Software Criticality

- How critical is the software to its stakeholders?
- The more critical → the more thorough the testing must be

Example: A fire alarm system requires extremely high confidence — a bug could cost lives. An LMS requires less — a bug is inconvenient but not dangerous.

2. User Expectations

- Users may have low expectations for newly installed systems
- They may tolerate early failures because benefits outweigh costs
- As software matures → users expect it to be more reliable
- Later versions of a system require more thorough testing than early releases

3. Marketing Environment

- Business pressure can affect testing thoroughness
- Getting a product to market early may be more important than finding every defect
- In a competitive market, a company may release before fully tested to beat competitors
- If software is very cheap → users may tolerate a lower level of reliability

Factor	Low Confidence Needed	High Confidence Needed
Software Criticality	LMS, simple app	Fire alarm, medical system
User Expectations	Newly released software	Mature, established software
Marketing Environment	Competitive, early release	Safety-critical, no shortcuts

Exam Takeaway: The aim of V&V; is to establish that the system is fit for purpose. How much testing is needed depends on 3 factors — software criticality, user expectations, and marketing environment.

Topic 7: Inspections and Testing — Static vs Dynamic

Two Approaches to Software Verification

1. Software Inspections — Static Verification

"Concerned with analysis of the static system representation to discover problems."

- You examine the code, documents, or design — without actually running the system
- Called static because the system is NOT executed
- Can be done before implementation is complete
- Can be applied to any representation: requirements, design, configuration data, test data, source code

2. Software Testing — Dynamic Verification

"Concerned with exercising and observing product behavior."

- The system is actually executed with test data
- You observe the operational behavior of the running system
- Called dynamic because the system is actually running
- Can only be done when there is executable code

Static vs Dynamic — Key Comparison

	Software Inspections	Software Testing
Type	Static Verification	Dynamic Verification
System executed?	No	Yes
When can it be done?	Before implementation	After code is written
What is examined?	Documents, code, design	Running system behavior

Are They Opposing Techniques?

"Inspections and testing are complementary and not opposing verification techniques. Both should be used during the V&V process."

Complementary means they complete each other — each covers what the other cannot. Inspections cannot check non-functional characteristics like performance and usability because the system is not running. Testing can check these because the system is actually executed.

Exam Takeaway: There are two verification approaches — Inspections (static) and Testing (dynamic). They are complementary — both must be used together. Inspections cannot check non-functional requirements.

Tonic 8: Software Inspections

"Inspections involve people examining the source representation with the aim of discovering anomalies and defects."

What are Software Inspections?

A group of people carefully read and analyze the code or documents to find anomalies and defects before they cause problems. No execution of the system is involved — purely human examination.

Key Characteristics

1. Does Not Require Execution

- Inspections can be done before implementation is complete
- Can be performed on partially written code
- Makes it a static V&V; technique

2. Can Be Applied to ANY Representation

- Requirements documents
- Design diagrams
- Configuration data
- Test data
- Source code

3. Proven to be Effective

- Sommerville states inspections are an effective technique for discovering program errors
- In many studies, inspections find more defects per hour than testing

What Do Inspectors Look For?

- Logic errors and design flaws
- Deviation from coding standards
- Poor programming style
- Missing requirements coverage
- Security vulnerabilities
- Portability and maintainability issues

Exam Takeaway: Software inspections are a static V&V; technique where people examine system representations to find anomalies and defects — without executing the system. They can be applied to any representation and are proven to be highly effective.

Tonic 9: Advantages of Inspections

Advantage 1: No Error Masking

"During testing, errors can mask (hide) other errors. Because inspection is a static process, you don't have to be concerned with interactions between errors."

- In testing, when one error occurs it can cause a chain of unexpected outputs — called error propagation
- You can never be sure if anomalies after the first error are new errors or side effects
- In inspections, errors are examined independently — no propagation possible
- A single inspection session can discover many errors without one masking another

Example: If a calculation error causes wrong output, subsequent display errors might just be consequences — testing cannot tell them apart. Inspection can.

Advantage 2: Incomplete Systems Can Be Inspected

"Incomplete versions of a system can be inspected without additional costs."

- If a program is incomplete, you cannot test it directly
- To test incomplete code you need specialized test harnesses — extra code simulating missing parts — which costs time and money
- With inspections, you can examine whatever exists — no test harnesses needed
- Makes inspections very useful early in development

Advantage 3: Checks Standards, Portability & Maintainability

- Inspections can verify the code complies with standards
- Can check portability — will the code work on different platforms?
- Can check maintainability — is the code easy to maintain?
- Can identify inefficient code and poor programming styles

Advantage 4: More Effective for Defect Discovery

- Sommerville states inspections are more effective for defect discovery than testing
- Especially useful for logic errors and design flaws

Limitation of Inspections

"Inspections cannot propose defects that arise because of interactions between systems — as the system has not been run yet."

- If a defect only appears when two systems interact at runtime — inspection will not catch it
- This is why testing is still needed alongside inspections

Advantage	Explanation
No error masking	Errors examined independently — no propagation
Incomplete systems	No test harnesses needed — inspect whatever exists
Standards & maintainability	Can check coding style, portability, compliance
More effective	Finds more defects per session than testing

Exam Takeaway: Inspections have 4 key advantages — no error masking, works on incomplete systems, checks standards/maintainability, and is more effective for defect discovery. However inspections cannot detect defects arising from runtime interactions.

Topic 10: Stages of Testing Overview

The 3 Stages of Testing

1. Development Testing

- Testing activities carried out during development by the development team
- Goal: discover bugs and defects early while the system is being built
- Includes: Unit testing, Component testing, System testing
- Done before the system is released

2. Release Testing

- A separate testing team tests a complete version of the system
- Done before the system is released to users
- Goal: check that the system meets the requirements of stakeholders
- The release testing team is independent from the development team

3. User Testing

- Users or potential users test the system in their own environment
- Done after the system is delivered to the customer
- Goal: verify the system works in real-world conditions with real users
- Acceptance testing is one type — customer formally decides whether to accept the system

Stage	Who Does It	When	Goal
Development Testing	Development team	During development	Find bugs early
Release Testing	Separate test team	Before release	Confirm requirements met
User Testing	Real users	After system delivered	Verify real-world behavior

Exam Takeaway: Testing is organized into 3 stages — Development Testing (by developers, during building), Release Testing (by separate team, before release), and User Testing (by real users, after system is delivered).

Tonic 11: Development Testing

"Development testing includes all testing activities that are carried out by the team developing the system."

Done during the development phase. The people closest to the code — developers and testers — are responsible. Goal: discover bugs and defects early before the system is released.

3 Approaches to Development Testing

1. Developers Testing Their Own Code

- The development team themselves test the software they wrote
- Most common approach in smaller projects
- Problem: developers may miss errors because they unconsciously test what they know works

2. Developers and Testers Working in Pairs

- A tester works alongside a developer simultaneously
- While the developer writes code, the tester is already writing and running test cases
- Catches defects much earlier — even as the code is being written
- Common in Agile development — closely related to Test-Driven Development (TDD)

3. Separate Testing Team (Safety-Critical Systems)

- A completely separate, dedicated testing team works alongside developers
- This team keeps a detailed record of every single test case run on the system
- Nothing is left undocumented
- The separation ensures no bias — testers have no attachment to the code

Example: Software for aircraft control systems, medical devices, nuclear plant systems

Approach	Used When	Key Feature
Developers test own code	Small/general projects	Fast but potentially biased
Developer-tester pairs	Agile projects	Simultaneous development & testing
Separate testing team	Safety-critical systems	Independent, fully documented

Exam Takeaway: Development testing is done by the development team during the building phase. 3 approaches exist — developers testing own code, developer-tester pairs, and separate testing teams for safety-critical systems.

Topic 12: Stages of Development Testing: Unit, Component & System

Stage 1: Unit Testing

"Individual program units or object classes are tested in isolation."

- Tests the smallest pieces of the system individually
- Focuses on testing the functionality of individual objects or methods
- Done in the development phase of SDLC
- It is a defect testing process

Example: Testing a single method like `calculateTotal()` or a single class like `BankAccount`

Stage 2: Component Testing

"Multiple individual units integrate to create composite components."

- After individual units are tested, they are combined together into components
- Focuses on testing compatibility and co-existence between units
- Checks: are components passing data correctly? Do dependent components work together?
- Done in the testing phase of SDLC

Example: Testing that a `Login` unit and a `Session Manager` unit work correctly together

Stage 3: System Testing

"Some or all components in a system are integrated and the overall system is tested."

- Tests the entire system as a whole
- Focuses on testing interactions between all components
- Checks that the integrated system behaves as expected end-to-end
- Done in the testing phase of SDLC

Example: Testing the entire banking application — login, transactions, reports, logout — all working together

	Unit Testing	Component Testing	System Testing
What is tested	Individual units/methods	Combined components	Entire system
Focus	Functionality of single unit	Compatibility & data transfer	Component interactions
SDLC Phase	Development phase	Testing phase	Testing phase

Exam Takeaway: Development testing has 3 stages — Unit testing (individual units, development phase), Component testing (combined units, checks compatibility, testing phase), and System testing (whole system, checks interactions, testing phase).

Topic 13: Unit Testing & Object Class Testing

"Unit testing is the process of testing individual components in isolation. It is a defect testing process."

Each unit is tested completely separately from the rest of the system. Done in the development phase of SDLC.

What Can a Unit Be?

Unit Type	Description	Example
Individual function/method	A single operation in an object	calculateSalary()
Object class	A class with several attributes and methods	BankAccount class
Composite component	A defined interface used to access functionality	Login module

What Does Unit Testing Check?

1. All Events That Can Change Object State

- Any event that modifies the state of an object must be tested

Example: For a BankAccount — deposit, withdrawal, and transfer all change account balance and must be tested

2. Individual Functions/Methods

- Every method in the unit must be tested

Example: Testing getBalance(), deposit(), withdraw() separately

3. Object Attributes

- All attributes should be checked — do they initialize correctly? Update correctly after operations?

Object Class Testing (for OO Systems)

In object-oriented systems, the unit being tested IS the object class. Object class testing is unit testing applied specifically to OO systems.

- Test all operations (methods) associated with an object
- Check all object attributes — initialization and updates
- Simulate all events that cause a state change

Challenge — Inheritance & Polymorphism

- Generalization (inheritance) makes object class testing more difficult
- Due to polymorphism, overridden methods in subclasses behave differently from parent class
- You must test inherited/overridden operations in ALL subclasses separately

Example: Animal.makeSound() tested in Animal, then must be tested again in Dog and Cat subclasses because each override behaves differently

Weather Station Example — State-Based Testing

- Test cases developed for methods like reportWeather() and reportStatus()
- Sometimes the sequence of test cases matters
- To test shutdown(), you first need to execute restart() to verify results
- Use a state model to identify sequences of state transitions

Example: Shutdown -> Running -> Shutdown

Exam Takeaway: Unit testing tests individual components in isolation. A unit can be a function, class, or composite component. Complete unit testing covers all methods, attributes, and state-changing events. Due to polymorphism, overridden methods must be tested in every subclass.

Tonic 14: Automated Testing

"Unit testing should be automated so that tests are run and checked without manual intervention."

Instead of a human manually running tests and checking results, code runs the tests automatically. The entire test suite can run within seconds. Sommerville recommends automated testing wherever possible.

How Does it Work?

- Different test automation frameworks are used
- These frameworks provide test classes to create specific test cases
- Example framework: JUnit for Java
- The framework runs all implemented tests and reports success/failure status via a GUI

The 3 Components of an Automated Test Case

1. Setup

- Initialize the system with the test case
- Define the inputs and expected outputs
- Prepare everything the test needs before it runs

Example: Create a BankAccount object with a balance of 1000

2. Call

- Execute the object or method being tested
- Actually call the function/method you want to test

Example: Call the withdraw(200) method on the BankAccount object

3. Assertion

- Compare the actual result with the expected result
- If assertion evaluates to TRUE → test PASSED
- If assertion evaluates to FALSE → test FAILED

Example: Assert that getBalance() now returns 800. True = PASS / False = FAIL

Manual Testing	Automated Testing
Human runs each test	Framework runs all tests automatically
Slow and time consuming	Entire suite runs in seconds
Prone to human error	Consistent and reliable
Hard to repeat exactly	Perfectly repeatable

Exam Takeaway: Automated testing uses frameworks like JUnit to run tests without manual intervention. Every automated test has 3 components — Setup (initialize inputs and expected outputs), Call (execute the method), and Assertion (compare actual vs expected). True = pass, False = fail.

Tonic 15: Choosing Unit Test Cases

"Testing is a time-consuming procedure so it is necessary to define effective test cases."

You cannot test every possible input — so you must be strategic about which test cases you choose.

What Makes a Test Case Effective?

1. Validation Test Case

- The test case should do what it is supposed to do when given correct input
- Confirms the component works correctly under normal conditions

Example: Given valid patient data → record is saved correctly in the database

2. Defect Test Case

- The test case should identify defects in the component
- Designed to expose incorrect behavior

Example: Given invalid patient data → system generates an error and exits gracefully — not crash or save bad data

Practical Example — Patient Record System

Test Case Type	What it Does
Validation test case	Takes valid inputs and saves them correctly in the database
Defect test case	Validates all input fields — in case of erroneous input, generates error and exits gracefully

Two Strategies for Choosing Test Cases

Strategy 1: Partition Testing

- Identify groups of inputs that have common characteristics
- All inputs in the same group should be processed in the same way
- Choose test cases from within each group — one per partition is sufficient
- These groups are called equivalence partitions

Example: If a method accepts positive numbers, negative numbers, and zero — each is a separate partition needing its own test case

Strategy 2: Guideline-Based Testing

- Use testing guidelines based on previous experience of common programmer errors
- Reflect known patterns of mistakes in similar components

Example: Always test with empty inputs, null values, boundary values — because bugs most commonly appear there

Exam Takeaway: Effective test cases either validate correct behavior or identify defects. Two strategies: Partition Testing (group inputs by common characteristics) and Guideline-Based Testing (use experience-based rules).

Topic 16: Equivalence Partitioning

"Input data and output results often fall into different classes where all members of a class are related. Each class is an equivalence partition."

Inputs that belong to the same partition are treated the same way by the program. So testing ONE value from a partition is enough to represent ALL values in that partition.

Why Equivalence?

All values within a partition are equivalent from the program's perspective — the program handles them all the same way. Testing one = testing all. This saves enormous time.

Boundary Values — Critical Point

"Test cases should be chosen from each partition. Better to check boundary values as they are generally atypical values."

- Boundaries are where partitions meet each other
- Bugs most commonly hide at boundaries between partitions

Example: If valid range is 1-100: test 0 (just below), 1 (first valid), 100 (last valid), 101 (just above)

Concrete Example from Slides

Program accepts 4-10 inputs which are 5-digit integers greater than 10,000.

Partitions for Number of Inputs:

Partition	Range	Example Test Value
Too few inputs	Less than 4	2
Valid inputs	4 to 10	7
Too many inputs	More than 10	12

Partitions for Value of Each Input:

Partition	Range	Example Test Value
Too small	Less than 10,000	9,999
Valid value	Greater than 10,000 (5 digits)	50,000
Too large	More than 5 digits	100,000

Boundary Values to Always Test:

- 3 inputs (one below minimum) and 4 inputs (minimum valid)
- 10 inputs (maximum valid) and 11 inputs (one above maximum)
- 9,999 (one below valid value) and 10,000 (boundary value)
- 99,999 (maximum valid 5-digit number)

Exam Takeaway: Equivalence partitioning divides inputs into partitions where all values are processed the same way — testing one value per partition is sufficient. Always test BOUNDARY VALUES between partitions as bugs most commonly appear there.

Tonic 17: Testing Guidelines

Testing guidelines are based on programmer experience of common errors. Sommerville presents two sets of guidelines.

Part 1: Guidelines for Testing Sequences, Arrays & Lists

1. Use Arrays of Different Sizes

- Don't just test with one fixed-size array
- Test with small, medium, and large arrays
- Why: Bugs often appear when array size changes — like buffer overflows or index errors

2. Test First, Middle, and Last Elements

- Derive tests so that the first, middle, and last elements of the array are accessed
- Why: Off-by-one errors are extremely common — bugs often hide at the start or end of arrays

3. Test Array of Length -1

- Test for an empty or invalid array (length of -1 or 0)
- Why: Programs often crash or behave unexpectedly when given an empty array

Part 2: Whittaker's General Testing Guidelines

James Whittaker's guidelines are broader — applicable to any type of system.

1. Force All Error Messages

- Choose inputs that force the system to generate ALL its error messages
- Every error message path must be tested
- Why: Error handling code is often poorly tested and full of bugs

2. Cause Input Buffer Overflow

- Design inputs that cause input buffers to overflow
- Give the system more input than it can handle
- Why: Buffer overflows are a common source of crashes and security vulnerabilities

3. Repeat the Same Input

- Repeat the same input or series of inputs numerous times
- Why: To verify system consistency — the system should give the same output every time for the same input
- If it doesn't — there is a serious bug

4. Force Invalid Outputs

- Design test cases that force invalid outputs to be generated
- Make the system produce results it shouldn't
- Why: Tests whether the system handles and reports its own invalid outputs correctly

5. Force Computation Results Too Large or Too Small

- Push calculations to extreme limits — results too large or too small for the system to handle
- Why: To identify the system's breaking point — where it overflows, underflows, or produces incorrect results

Example: Integer overflow — a counter that keeps incrementing until it exceeds maximum integer value and wraps around to negative

Guideline Set	Focus	Best For
---------------	-------	----------

Sequence/Array Guidelines	Data structure testing	Arrays, lists, sequences
Whittaker's Guidelines	General system behavior	Any type of system

Exam Takeaway: For sequences/arrays — test different sizes, first/middle/last elements, and empty arrays. Whittaker's guidelines say — force all error messages, overflow buffers, repeat inputs, force invalid outputs, and push computations to extremes to find the system's breaking point.

Quick Revision Summary

Topic	Title	Key Point
1	What is Testing?	Sample data; 1 test per requirement (custom), test all features (generic)
2	Manual vs Automated & V&V	JUnit automates; Validation=right product, Verification=built right; Black/White box
3	Functional & Non-Functional	Functional=what it does; Non-functional=how well it does it
4	Validation & Defect Testing	Validation: success=works; Defect: success=system fails & exposes bug
5	Testing Process Goals	Formal definitions of validation and defect testing goals
6	V&V Confidence	Fit for purpose; depends on criticality, user expectations, marketing
7	Inspections vs Testing	Static (no execution) vs Dynamic (execution); complementary not opposing
8	Software Inspections	People examine representations; static V&V; any representation; proven effective
9	Advantages of Inspections	No error masking; incomplete systems; standards; more effective than testing
10	Stages of Testing	3 stages: Development, Release, User (after delivery)
11	Development Testing	3 approaches: own code, pairs, separate team (safety-critical)
12	Unit/Component/System	Unit=isolation(dev phase); Component=compatibility; System=interactions
13	Unit & Object Class Testing	All methods, attributes, state changes; polymorphism = retest subclasses
14	Automated Testing	JUnit; 3 parts: Setup, Call, Assertion; True=pass, False=fail
15	Choosing Test Cases	Effective = validates OR finds defects; Partition testing & Guideline-based
16	Equivalence Partitioning	One test per partition; always test boundary values
17	Testing Guidelines	Arrays: size/first-mid-last/empty; Whittaker: errors/overflow/repeat/invalid/extremes